Copyright © 2023. Bentham Science Publishers. All rights reserved.

Modules and Packages

Krishna Kumar Mohbey[1], **Malika Acharya**[1]

[1] Computer Science, Central University of Rajasthan, India

Abstract

Useful codes are often stored as separate files to increase modularity and reusability. Modules refer to a single file of code while a package is a collection of modules. A good programmer utilises both these aspects to enhance the program view and manage the hierarchy. In this chapter, we introduce the basics of working with the modules and packages.

Keywords: Scoping, Modular programming, Standalone script.

Introduction

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Modules

A complicated and unmanageable program can be broken down into several manageable subprograms, each referred to as a **module** through modular programming. One activity can be carried out by utilizing each component individually. The creation of modules in Python may be accomplished by utilizing Python files, which can include a variety of *and* statements. It is possible to define variables, classes, and functions in a module. A module can also make use of code that is executable. When the code is divided into modules, it is much simpler to comprehend and more convenient to utilize. Additionally, it logically arranges the code.

To put it another way, the file containing our Python source code with the extension (.py) is considered the module. Python modules can store code that can be executed. When we want to use the features of one module in another, we must import the specific module first. Let's say you've generated a file on your computer named **module1.py**, and inside it, you have the following code:

Filename: module1.py

```
1.  #(module1.py)
2.  def add_value (x, y):
3.   return (x+y)
4.  def sub_value (x, y):
5.   return (x-y)
```

To call the functions *add_value()* and *sub_value()* specified in the module named file (**module1.py**), we must include this module in our main module. To use the module's functionality, we must first load it into our Python code. Python has ***import*** and ***from..import*** statement to include a module.

Import Statement

Our Python program can connect to a module by utilizing an import statement. The import of many modules may be accomplished with a single import line; however, despite the number of times a module has been imported into our register, it is only loaded once each time. The syntax of the import statement is as follows:

Syntax:

```
import module1, module2, module3, ..... module n
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

When an import statement is found the interpreter imports the module specified within the search path. The interpreter explores every directory in the search path when importing a module. For instance, add the following line to the program's top to import the module **module1.py**.

Example:

```
1.  # importing module module1.py
2.  import module1
3.  print (add_value (2, 4))
4.  print (sub_value (10, 50))
```

Output:

```
6

60
```

From…Import Statement

Python enables users to import only a module's specified properties into the namespace rather than the entire module. For this, the from…import statement may be utilized. The following syntax makes use of the ***from...import*** expression.

Syntax:

```
from <module_name> import <name_1>, <name_2>….<name_n>
```

Consider the following module, **module 1**, which includes the functions *add_value()* and *sub_add()*.

Example:

Filename: module1.py

```
1.  # module1.py
2.  def add_value(x, y):
3.   return (x+y)
4.  def sub_value(x, y):
5.   return (x-y)
```

If we want to import only *add_value()* function from this module, then the following code will be used.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Filename: main.py

```
1.  # importing add  value()from module1.py
2.  from module1 import add_value()
3.  print (add_value(100, 200))
```

Output:

```
300
```

We can also import any built-in module in our program. The below example imports the **pi** function from the *math* module.

Example:

```
1.  #importing pi from the math module
2.  from math import pi
3.  print ("pi value=", pi)
```

Output:

```
pi value= 3.141592653589793
```

The **from...import** statement may be used if we know the properties that need to be imported from the module in advance. * may also be used to import all of a module's attributes, which won't make our code any larger. Look at the example below, where all *module1's* functions are imported.

Example:

Filename: main.py

```
1.  # importing all attributes from module1.py
2.  from module1 import *
3.  print (add_value(10, 20))
4.  print (sub_value(100, 50))
```

Output:

```
30

50
```

Renaming A Module

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Python allows us to import modules with specific names and then use those names to refer to those modules in our source code. To rename a module in Python code ***as*** keyword is used. The syntax for renaming a module is as follows:

Syntax:

```
import <module-name> as <new-name>
```

Example:

```
1.  # importing module1.py
2.  import module1 as mm    # rename module1 to mm
3.  print (mm.sub_value(20, 10))
4.  print (mm.add_value(100, 20))
```

Output:

```
>>> dir(module1)
['__builtins__',
'__cached__',
'__doc__',
'__file__',
'__initializing__',
'__loader__',
'__name__',
'__package__',
'add_value',
'sub_value']
```

Dir() Built-In Function

The *dir()* function can be used to find names specified within a module. In module *module 1*, for example, we've specified two functions: *add_value()* and *sub_value()*. In the *module1* module, we can use *dir()* as follows:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
>>> dir(module1)
['__builtins__',
'__cached__',
'__doc__',
'__file__',
'__initializing__',
'__loader__',
'__name__',
'__package__',
'add_value',
'sub_value']
```

Here, we can see a list of names that have been sorted (along with *add_value* and *sub_value*). The module's default Python characteristics may be identified by their names, beginning with an **underscore (_)** character. Finding all of the names that are specified in our current namespace may be accomplished by using the *dir()* function without passing any parameters to it.

Example:

```
> import math
> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__',
'__spec__', 'math']
```

Reload () Function

As stated earlier, a module is only loaded once, regardless of how often it is imported into the Python source file. However, Python gives us the *reload()* function to reload a module that has already been imported and rerun the top-level code. The following syntax takes advantage of the *reload()* method.

Syntax:

```
reload(<module-name>)
```

Example:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
reload(module1)
```

Built-In Modules

In addition to built-in functions, libraries included with Python releases also offer many pre-defined functions. Built-in modules describe the tasks that are specified in modules. Built-in modules developed in C are incorporated into the Python shell. Resources for system-specific tasks like OS administration, disc I\O, and other similar tasks are included in each built-in module. The standard library also contains many Python scripts with the ***.py*** extension that provide practical utility. To view a complete listing of all available modules, the following command is used:

```
help('modules')
```

Some commonly used built-in modules include:

1. OS Module: It facilitates OS-related operations like, create folders, delete folder, fetch the contents, change, and identify the current directory.
2. Sys Module: It allows modification of different parts Python runtime environment.
3. Math Module: This module supports mathematical functions like trigonometry, angle conversion, logarithmic function, *etc*.
4. Statistics module: It supports the statistical operation over numeric data.
5. Collection Module: It provides an alternative to built-in container data types like lists, tuple, and dict. It supports OrderedDict(), nametuple(), deque(), *etc*.
6. Random Module: It is used to generate pseudo-random variables.

Example:

```
1.  import math
2.  #natural log
3.  print("natural log:", math.log(45))
4.  #standard log
5.  print("standard log:", math.log10(84))
6.  #exponential
7.  print("exponential:", math.exp(9))
8.  #power function
9.  print("power value:",math.pow(3,4))
10. #ceiling
11. print("ceiling function:", math.ceil(6.78954))
12. #floor
13. print("floor function:",math.floor(1.544847))
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Output:

```
natural log: 3.8066624897703196
standard log: 1.9242792860618816
exponential: 8103.083927575384
power value: 81.0
ceiling function: 7
floor function: 1
```

There is a large number of built-in modules. The below example shows the uses of the datetime built-in module.

Example:

```
1.  from datetime import date
2.  # date object for current date
3.  today = date.today()
4.  print("Current year:", today.year)
5.  print("Current month:", today.month)
6.  print("Current day:", today.day)
7.
8.  # Datetime from timestamp
9.  date_time = datetime.fromtimestamp(19745274674)
10. print("Datetime from timestamp:", date_time)
```

Output:

```
Current year: 2023
Current month: 1
Current day: 5
Datetime from timestamp: 2595-09-14 06:31:14
```

PACKAGE

To make finding and managing many files simpler, we classify them into different categories and subfolders depending on predetermined criteria. However, a module in Python takes the idea of modularity to its logical conclusion. As you may already know, a module can include many objects, including classes, functions, *etc*. In a package, one or more related modules may be present. A

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

package is a folder that physically contains one or more module files. Similar to how discs and directories enable us to store data in an operating system, packages help us store additional sub-packages and modules that the user may utilize at any time.

Creating Package

We build a file called **__init__.py** within a directory to inform Python that it is a package, and then it is considered a package, and we can create other modules and sub-packages inside it. This **__init__.py** file may be left blank or filled in with the package's initialization code.

We can follow these three basic steps to build a package in Python:

1. First, we create a directory and name it a package, preferably one related to its function.
2. After that, we add the required classes and functions.
3. Finally, inside the directory, we construct an **__init__.py** file to inform Python that it is a package.

Example:

1. Make a directory called **Details**.

2. After that, we must build modules.

a. Create a file called **program.py** and write the below code.

Filename: program.py

```
1. class program:
2.     def __init__(self):
3.         self.data1 = ['BTech', 'MSC', 'IMSc']
4.
5.     def show(self):
6.         print("Program details:")
7.
8.         for s in self.data1:
9.             print(s)
```

b. Create a file called **course.py** and write the below code.

Filename: course.py

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
class course:
    def __init__(self):
        self.data2 = ['TOC ', 'ADA', 'CN']

# function
    def show(self):
        print('course details)
        for s in self.data2:
            print(s)
```

c. Create a file called **duration.py** and write the below code.

Filename: duration.py

```
class duration:
    def __init__(self):
        self.data3 = [4, 2, 5]
#function
    def show(self):
        print('duration')
        for s in self.data3:
            print(s)
```

3. Create **__init__.py** file in the **Details** directory and either leave it blank or fill it with the initialization code.

Filename: __init__.py

```
from program import program
from course import course
from duration import duration
```

4. Create a **Test.py** file in the same directory as the **Details** package and write the following code:

Filename: Test.py

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
#Import classes from the Details Package
from Details import program
from Details Import course
from Details import duration

# Create an object of program class
obj1 = program()
obj1.show()

# Create an object of course class
obj2 = course()
obj2.show()

# Create an object of duration class
obj3 =duration()
obj3.show()
```

Output:

```
Program details:
BTech
MSC
IMSc
course details
TOC
ADA
CN
duration
```

Fig. (1) depicts the hierarchy of modules in the package.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

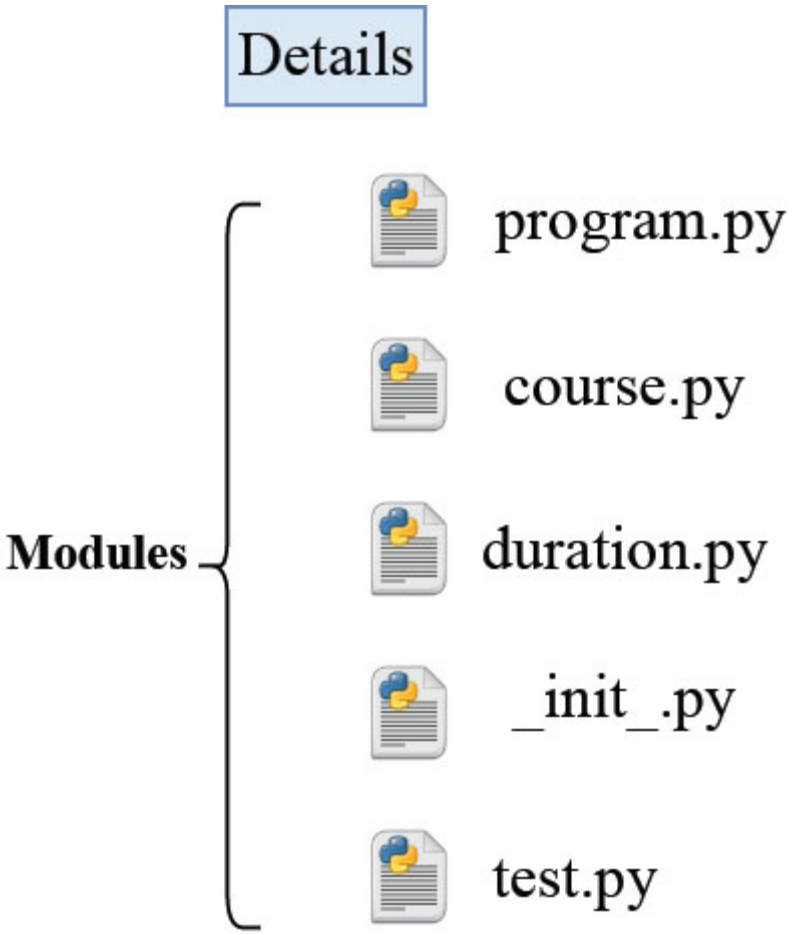


Fig. (1))

Hierarchy of Modules in the Details package.

__init__.py

The content of the package is stored in a particular file named **__init__.py**, which is found in the package folder. It has two functions:

1. If a folder contains the *__init__.py* file, the Python interpreter recognizes it as a package.
2. *__init__.py* makes resources from its modules available for import.

When this package is imported, an empty *__init__.py* file makes all modules' functions accessible. It is worth noting that the folder's *__init__.py* file is needed for Python to recognize it as a package.

Sub Packages

The package hierarchy detailed above can have additional nested packages within the same package. This structural hierarchy aids in program management as one can accentuate different

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

modules properly. Further, the division into different modules reduces the length of the program and makes it easy to handle. Code modification requires less time as only the required module can be modified and not the whole program.

Fig. (2) is the specimen of the nested packages. The **Details** package comprises two other sub-packages namely, **Subpackage1** and **Subpackage2.** Each of these packages further encompasses the module files. As in the previous example, there are three modules namely, ***program.py, course.py, duration.py, _init_.py,*** and ***test.py.*** Each of these modules is further encapsulated in two sub-packages, rather than in a single main package. The package hierarchy doesn't complicate the import process much. Now for import purposes, you have to specify the name of the subpackage as well alongside the main package separated by the dot (.) as shown in the example.

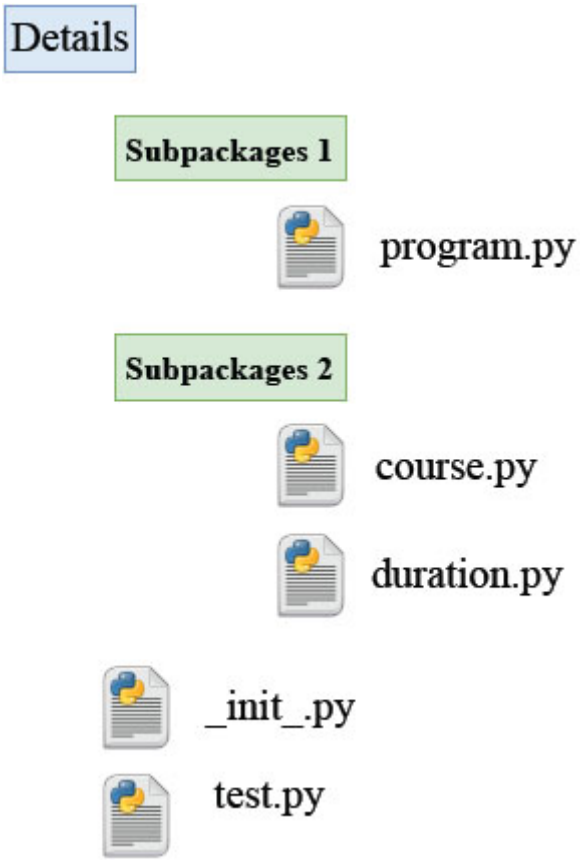


Fig. (2))

Details package structure with sub-packages.

Example:

```
1.  Import Details.Subpackage1.program
2.  import Details.Subpackage2.course
3.  import Details.Subpackage2.duration
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

CONCLUDING REMARK

We covered Python modules and packages in this chapter. Additionally presented are module implementations and their importing. Furthermore, Python codes are used to discuss package creation and services. You can write codes based on these examples. We will talk about Python's object-oriented programming concepts in the following chapter.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-25 19:51:23.